

Full Stack Development with MERN

Project Documentation

1. Introduction

- **Project Title:** BookStore
- **Team Members:** Mandapati Madhav Krishna Yadav – Database
M.Akhila – Frontend
M.N Nandhini – Backend

2. Project Overview

- **Purpose:** The Online BookStore Management System is a MERN stack web application that enables users to browse and purchase books online. Sellers can manage their book inventory, while administrators oversee users, books, and orders through a dedicated dashboard.
- **Features:**
 - User Registration & Login
 - JWT Authentication
 - Browse Books
 - View Book Details
 - Place Orders
 - My Orders
 - Seller Dashboard
 - Add & Delete Books
 - Upload Book Images
 - Admin Dashboard
 - Manage Users
 - Manage Books
 - Manage Orders

3. Architecture

- **Frontend:** The frontend is developed using React.js with React Router for navigation, Bootstrap for responsive UI, Axios for API communication, and Context API for authentication state management.
- **Backend:** The backend is developed using Node.js and Express.js, providing RESTful APIs for authentication, book management, seller operations, admin operations, and order processing.
- **Database:** MongoDB is used as the database. Mongoose is used to define schemas for:
 - Users
 - Books
 - Orders

4. Setup Instructions

- **Prerequisites**
 - Node.js
 - npm
 - MongoDB
 - Git

- VS Code
- **Installation:**
 - Clone the repository
git clone <https://github.com/MadhavMk125/BookStore-MERNPROJECT.git>
 - Backend
cd server
npm install
npm run dev
 - Frontend
cd client
npm install
npm run dev

5. Folder Structure

- **Client:**

```

client/
├── src/
│   ├── components/
│   ├── context/
│   ├── pages/
│   ├── services/
│   ├── App.jsx
│   └── main.jsx

```
- **Server:**

```

server/
├── config/
├── controllers/
├── middlewares/
├── models/
├── routes/
├── uploads/
└── server.js

```

6. Running the Application

- **Backend**
cd server
npm run dev
- **Frontend**
cd client
npm run dev
- **Application URLs**
Frontend:<http://localhost:5173>
Backend:<http://localhost:8000>

7. API Documentation

Seller APIs

Method	Endpoint
GET	/api/seller/books

Book APIs

Method	Endpoint
GET	/api/books
GET	/api/books/:id
POST	/api/books/add
PUT	/api/books/update/:id
DELETE	/api/books/delete/:id

Order APIs

Method	Endpoint
POST	/api/orders/place
GET	/api/orders/myorders

Admin APIs

Method	Endpoint
GET	/api/admin/dashboard
GET	/api/admin/users
GET	/api/admin/books
GET	/api/admin/orders

User APIs

Method	Endpoint
POST	/api/users/register
POST	/api/users/login

8. Authentication

Authentication is implemented using **JWT (JSON Web Token)**.

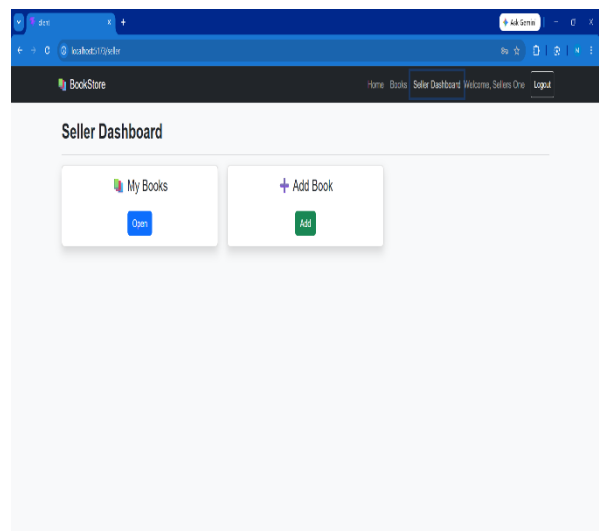
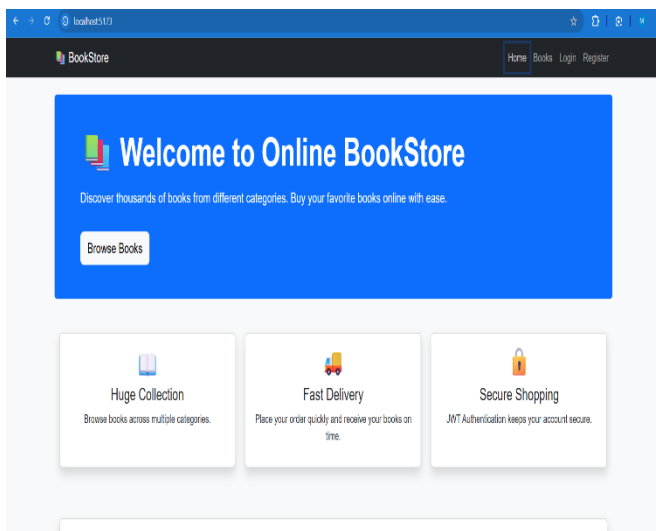
Security features include:

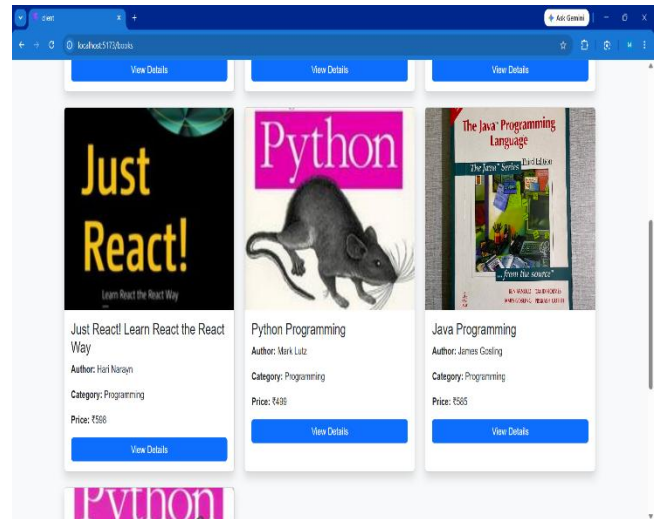
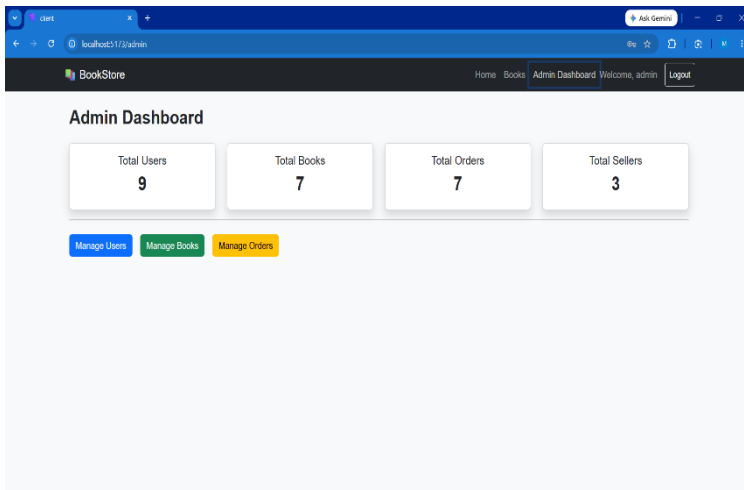
- Password hashing using bcrypt
- JWT token generation
- Protected Routes
- Role-based authorization (User, Seller, Admin)

9. User Interface

The application provides separate interfaces for three roles:

- User Interface
- Seller Dashboard
- Admin Dashboard





10. Testing

Testing was performed manually using:

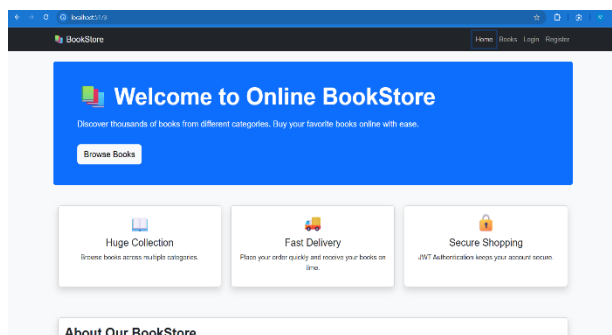
- Browser testing
- Postman API testing

The following functionalities were verified:

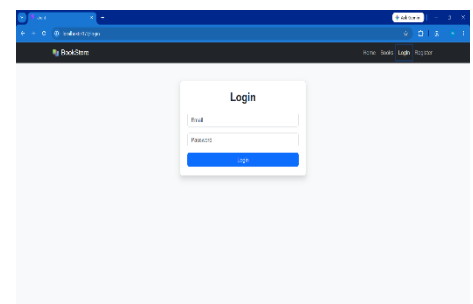
- User Registration
- User Login
- Book Management
- Order Placement
- Seller Operations
- Admin Operations
- Image Upload
-

11. Screenshots or Demo

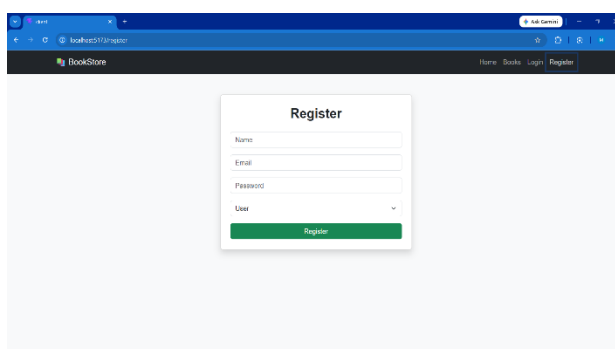
- Home page



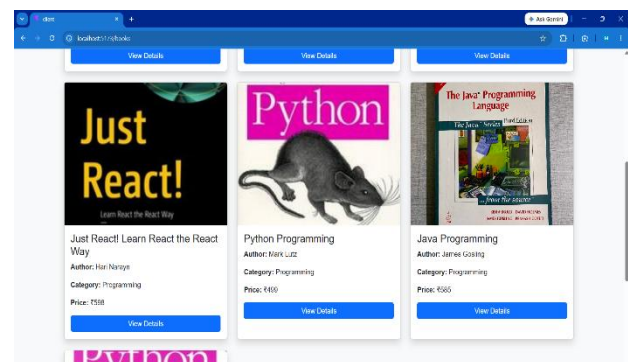
Login Page



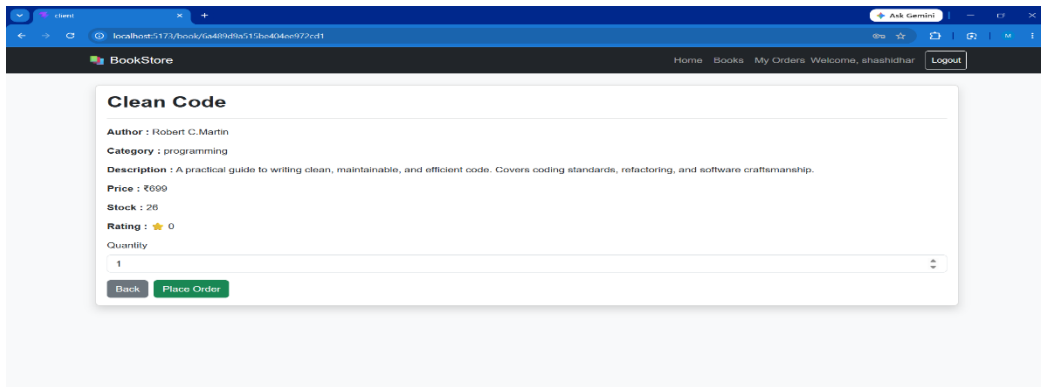
- Registration Page



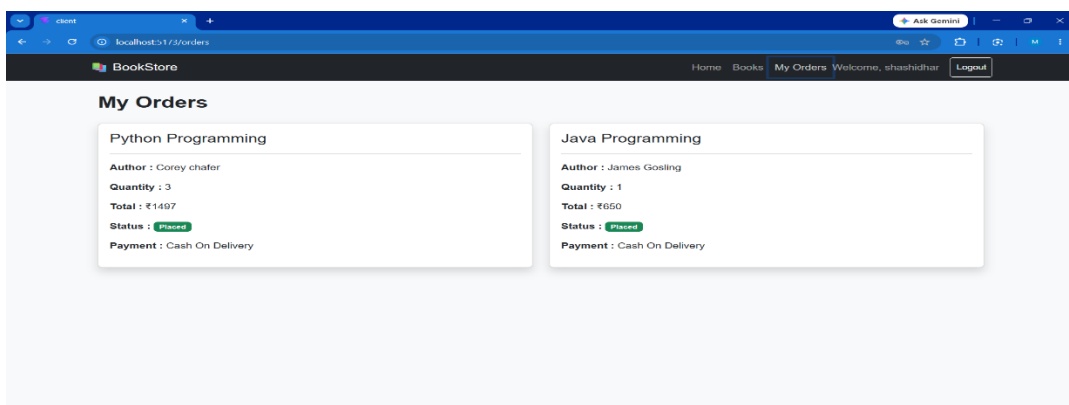
Books page



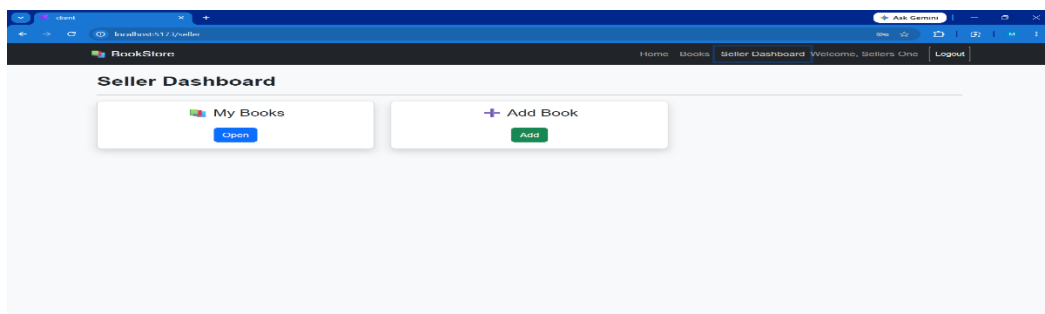
○ Book Detail



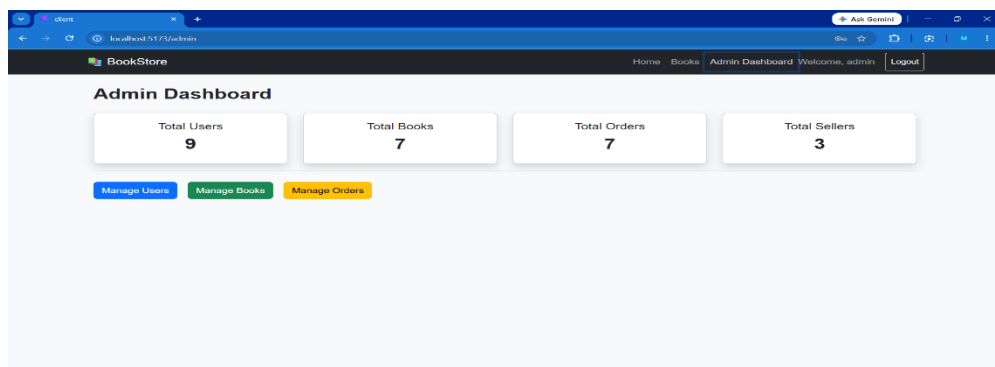
○ My Order



○ Seller Dashboard



○ Admin Dashboard



12.Known Issues

- Online payment gateway is not implemented.
- Book reviews are currently limited.
- Search functionality can be enhanced.
- Email notifications are not available.

13.Future Enhancements

- Online Payment Integration
- Wishlist Feature
- Book Recommendation System
- Advanced Search Filters
- Email Notifications
- Cloud Image Storage
- Mobile Responsive Enhancements.